

Part 1: Conceptual Questions

- **Q1:** What is the purpose of `{% csrf_token %}` in the template files?
 - The purpose of `{% csrf_token %}` is to protect against Cross Site Request Forgery by requiring the user to have a valid `csrf_token` when making a certain request.
- **Q2:** What could happen if a developer forgets to add the `{% csrf_token %}` tag on their template that uses an HTML form?
 - If the user visits a malicious site, the malicious site may make requests and rely on session cookies for the application to recognize the user as the one making the requests.
- **Q3:** What is the CWE ID associated with the security problem that arises with forgetting to use `{% csrf_token %}`?
 - CWE-352

Part 2: Finding Issues

1. Vulnerability #1:
 - a. On line 37 of `views.py`, we have the following line of code:
`cursor.executescript(f"INSERT INTO tasktracker_task(user_id, status, due_date, title) VALUES ({request.user.id}, '{status}', '{due_date}', '{title}'))"`
 - b. Since the application is interacting with the SQL database directly, and not using Django's built in features, you can exploit this by adding in a malicious code in the title. Start the code with `"*"` where `*` is the title itself and doesn't matter, and the `'` closes the tag. Afterwards, you can add malicious code that will still be read by the `cursor.executescript()` function. For example, you could very easily add code to Truncate a specific table, and when it is read by the `executescript()` function, it will delete the data in that table.
 - c. CWE-89: SQL Injection
2. Vulnerability #2:
 - a. `views.py` and `login.html`
 - b. Since the application does not enforce any form of rate limiting, an attacker could very easily try and brute force the passcode for a particular user by trying every possible combination of (reasonable) characters.
 - c. CWE-30: Improper Restriction of Excessive Authentication Attempts
3. Vulnerability #3:
 - a. On line 29 of `settings.py`, we have the following line of code: `DEBUG = True`
 - b. With this setting on, error pages can reveal things such as your settings, environment variables, the `SECRET_KEY`, passwords, and more. With this information, attackers can do any number of things.
 - c. CWE-489: Active Debug Code
4. Vulnerability #4:

- a. On line 11 of index.html, we have the following line of code: `<b>Task #{{t.id}}: {{t.title | safe}}</b>`
- b. The `| safe` disables Django's automatic escaping (i.e. it escapes characters like `"<"`, `">"`, etc.). An attacker can create a task where the title contains `<script>` `</script>` and have the server execute malicious code.
- c. CWE-79: Cross-Site Scripting (XSS)

Part 3: Cracking Passwords

See the other files included in the zip

1) Would Pythia help you in finding the vulnerabilities you identified in Part 2?

*The answer will be **at least** half a page, explaining in details how Pythia could have helped you in finding vulnerabilities in Part 2. In case you think the tool could not help you, explain in details why it is the case (i.e., what is it on this tool that makes it unsuitable to find the vulnerabilities you have found).*

Part 4: Research Synthesis

1. Web frameworks such as Django provide several built-in security mechanisms such as escaping html, and utilizing CSRF tokens. However, these security mechanisms can limit developers to write and implement certain features. As such, often times developers will disable these security mechanisms, leaving their application susceptible to classic vulnerabilities such as Cross-Site Scripting (XSS) or Cross-Site Request Forgery (CSRF). Likewise, detecting these vulnerabilities can be difficult because templates can often inherit and include other templates and the different components of an application can interact with each other in complex ways.

To tackle this problem, the researchers developed Pythia for analyzing django projects specifically. It does this by first identifying sinks. Specifically, template sinks and In-view sinks. Template sinks involve Django filters such as `safe` and `safeseq`, and tags such as `autoescape`. In-view sinks include decorators such as `@csrf_exempt` and functions invocations that can lead to xss defects when used in an improper manner. Afterwards, it attempts to trace data flows from potential sources (such as user input) through views into sinks. To do this, it utilizes Abstract Syntax Trees (AST) analysis to track inheritance among templates.

2. Yes, Pythia would be very helpful in finding the vulnerabilities identified in part two. Perhaps the clearest instance of where Pythia would be helpful would be in identifying the SQL Injection vulnerability in views.py. This vulnerability would surely be identified when Pythia attempts to trace dangerous data flows. It would recognize that any user input (especially text as is the case with "title") may be treated as tainted. As such, it

would follow that data as it moves through the application. It would be able to do this by tracking the inheritance of the tainted data as it moves from add.html (where it was collected) to views.py, where an unsafe operation is conducted with the tainted data.

Likewise, Pythia would also be able to easily identify the Cross-Site Scripting (XSS) vulnerability in index.html. This would likely be recognized when Pythia is inspecting Template sinks. It would see that the Django filter “safe” is being utilized, and recognized that it could lead to tainted data, making the application vulnerable. It should also be noted that Pythia’s limitations likely wouldn’t be a major issue when examining this application. The chances of it encountering a false negative are low since the application is not invoking helper functions. Likewise, the chances are low that it will report a false positive given that it will likely not have to be able distinguish between trusted and un-trusted data.